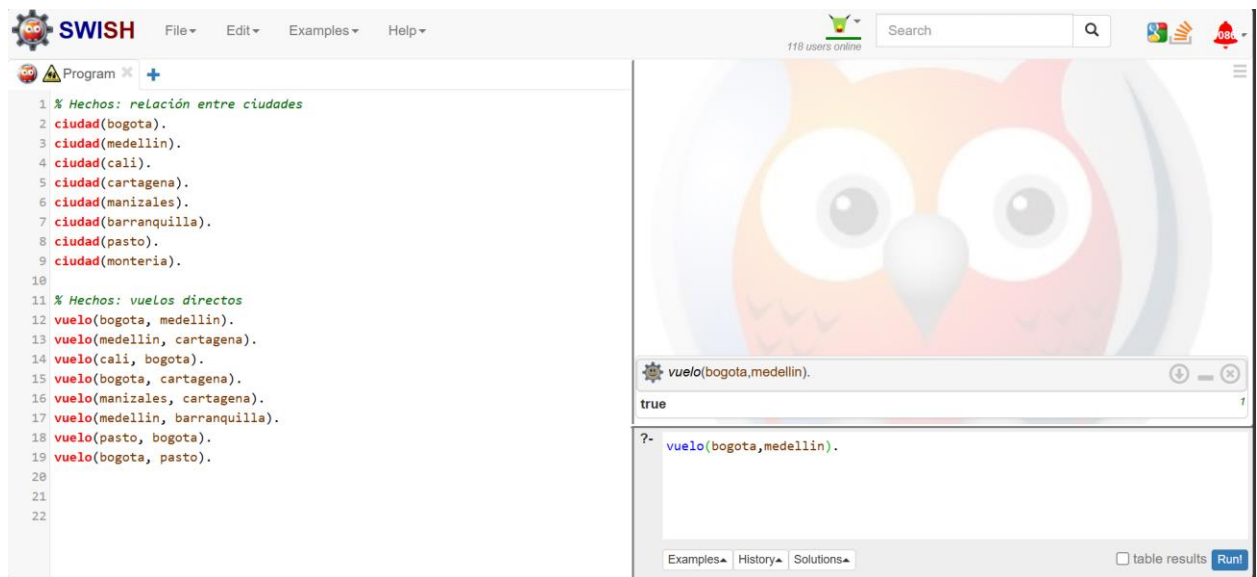


Laboratorio 01 - Introducción a Prolog

Ejercicio 1

1. ¿Existe un vuelo directo de Bogotá a Medellín?

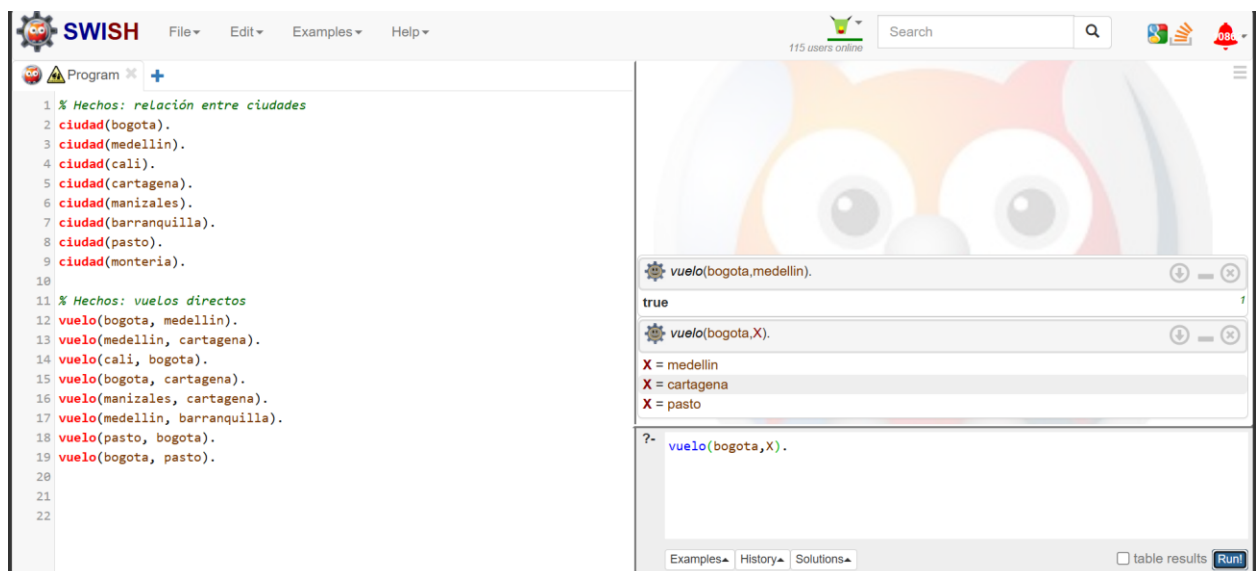


The screenshot shows the SWISH Prolog environment. The left pane contains the following Prolog code:

```
1 % Hechos: relación entre ciudades
2 ciudad(bogota).
3 ciudad(medellin).
4 ciudad(cali).
5 ciudad(cartagena).
6 ciudad(manizales).
7 ciudad(barranquilla).
8 ciudad(pasto).
9 ciudad(monteria).
10
11 % Hechos: vuelos directos
12 vuelo(bogota, medellin).
13 vuelo(medellin, cartagena).
14 vuelo(cali, bogota).
15 vuelo(bogota, cartagena).
16 vuelo(manizales, cartagena).
17 vuelo(medellin, barranquilla).
18 vuelo(pasto, bogota).
19 vuelo(bogota, pasto).
20
21
22
```

The right pane shows the query `vuelo(bogota,medellin).` with the result `true`. Below the query, there is a prompt `?- vuelo(bogota,medellin).` and a `Run!` button.

2. ¿Qué destinos se pueden alcanzar directamente desde Bogotá?



The screenshot shows the SWISH Prolog environment. The left pane contains the same Prolog code as in the previous screenshot. The right pane shows the query `vuelo(bogota,X).` with the result `true`. Below the query, there is a prompt `?- vuelo(bogota,X).` and a `Run!` button. The results of the query are displayed as a list of destinations:

- X = medellin
- X = cartagena
- X = pasto

3. ¿Desde que destinos se puede alcanzar Medellin?

The screenshot shows the SWISH Prolog IDE interface. The left pane contains the following Prolog code:

```
1 % Hechos: relación entre ciudades
2 ciudad(bogota).
3 ciudad(medellin).
4 ciudad(cali).
5 ciudad(cartagena).
6 ciudad(manizales).
7 ciudad(barranquilla).
8 ciudad(pasto).
9 ciudad(monteria).
10
11 % Hechos: vuelos directos
12 vuelo(bogota, medellin).
13 vuelo(medellin, cartagena).
14 vuelo(cali, bogota).
15 vuelo(bogota, cartagena).
16 vuelo(manizales, cartagena).
17 vuelo(medellin, barranquilla).
18 vuelo(pasto, bogota).
19 vuelo(bogota, pasto).
20
21
22
```

The right pane shows a query window with the query `vuelo(X,medellin).` and the result `X = bogota`. Below the query window, there is a section for running the query with a `Run` button and a checkbox for `table results`.

1. 4. ¿Hay alguna forma de llegar directamente a cali?

The screenshot shows the SWISH Prolog IDE interface. The left pane contains the same Prolog code as the previous screenshot:

```
1 % Hechos: relación entre ciudades
2 ciudad(bogota).
3 ciudad(medellin).
4 ciudad(cali).
5 ciudad(cartagena).
6 ciudad(manizales).
7 ciudad(barranquilla).
8 ciudad(pasto).
9 ciudad(monteria).
10
11 % Hechos: vuelos directos
12 vuelo(bogota, medellin).
13 vuelo(medellin, cartagena).
14 vuelo(cali, bogota).
15 vuelo(bogota, cartagena).
16 vuelo(manizales, cartagena).
17 vuelo(medellin, barranquilla).
18 vuelo(pasto, bogota).
19 vuelo(bogota, pasto).
20
21
22
```

The right pane shows a query window with the query `vuelo(X,cali).` and the result `false`. Below the query window, there is a section for running the query with a `Run` button and a checkbox for `table results`.

Ejercicio 2

- Determine si dos ciudades están conectadas mediante una escala.
 - Ahora, verifica si existe una conexión de Bogotá a Barranquilla.

```
escala(X,Y):-vuelo(X,Z),vuelo(Z,Y).
```

The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
1 % Hechos: relación entre ciudades
2 ciudad(bogota).
3 ciudad(medellin).
4 ciudad(cali).
5 ciudad(cartagena).
6 ciudad(manizales).
7 ciudad(barranquilla).
8 ciudad(pasto).
9 ciudad(monteria).
10
11 % Hechos: vuelos directos
12 vuelo(bogota, medellin).
13 vuelo(medellin, cartagena).
14 vuelo(cali, bogota).
15 vuelo(bogota, cartagena).
16 vuelo(manizales, cartagena).
17 vuelo(medellin, barranquilla).
18 vuelo(pasto, bogota).
19 vuelo(bogota, pasto).
20
21 escala(X,Y):-vuelo(X,Z),vuelo(Z,Y).
22
23
```

The right pane shows the execution of the query `escala(bogota,barranquilla).`. The result is `true`. Below the result, there is a section for the query `?- escala(bogota,barranquilla).` with a `Run!` button.

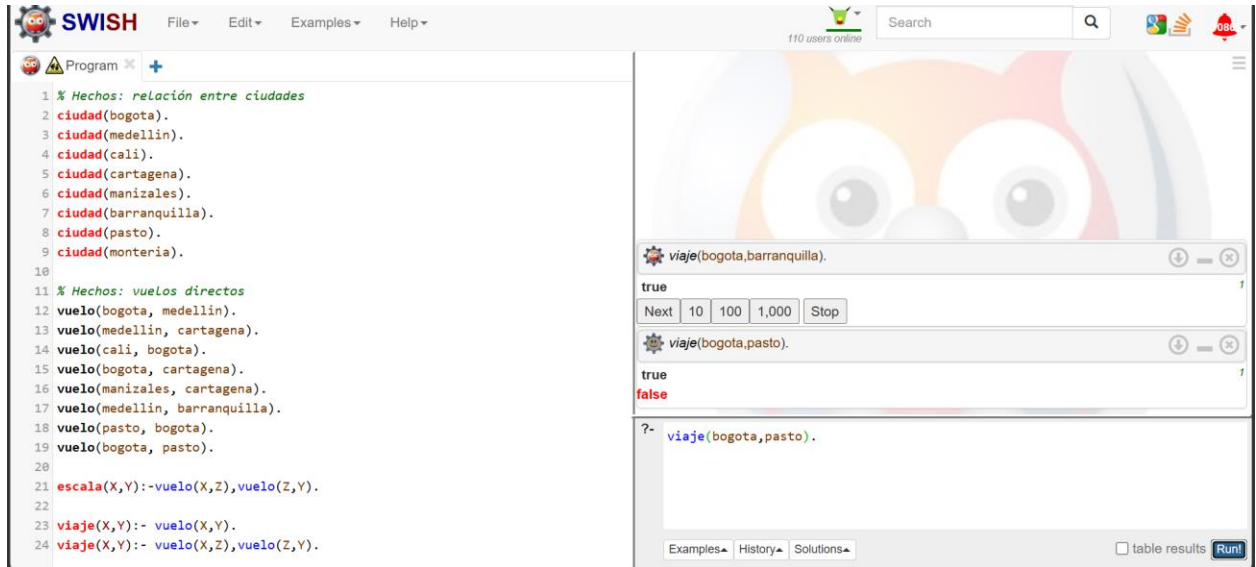
Encuentra todas las ciudades a las que se puede llegar desde Cali con una escala.

The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
1 % Hechos: relación entre ciudades
2 ciudad(bogota).
3 ciudad(medellin).
4 ciudad(cali).
5 ciudad(cartagena).
6 ciudad(manizales).
7 ciudad(barranquilla).
8 ciudad(pasto).
9 ciudad(monteria).
10
11 % Hechos: vuelos directos
12 vuelo(bogota, medellin).
13 vuelo(medellin, cartagena).
14 vuelo(cali, bogota).
15 vuelo(bogota, cartagena).
16 vuelo(manizales, cartagena).
17 vuelo(medellin, barranquilla).
18 vuelo(pasto, bogota).
19 vuelo(bogota, pasto).
20
21 escala(X,Y):-vuelo(X,Z),vuelo(Z,Y).
22
23
```

The right pane shows the execution of the query `escala(cali,X).`. The result is `X = medellin`, `X = cartagena`, and `X = pasto`. Below the result, there is a section for the query `?- escala(cali,X).` with a `Run!` button.

- Define una regla viaje que sea cierta si existe un vuelo directo o con una escala entre dos ciudades (no usar recursion).
 - Ahora, verifica si existe un viaje posible de Bogotá a Pasto.

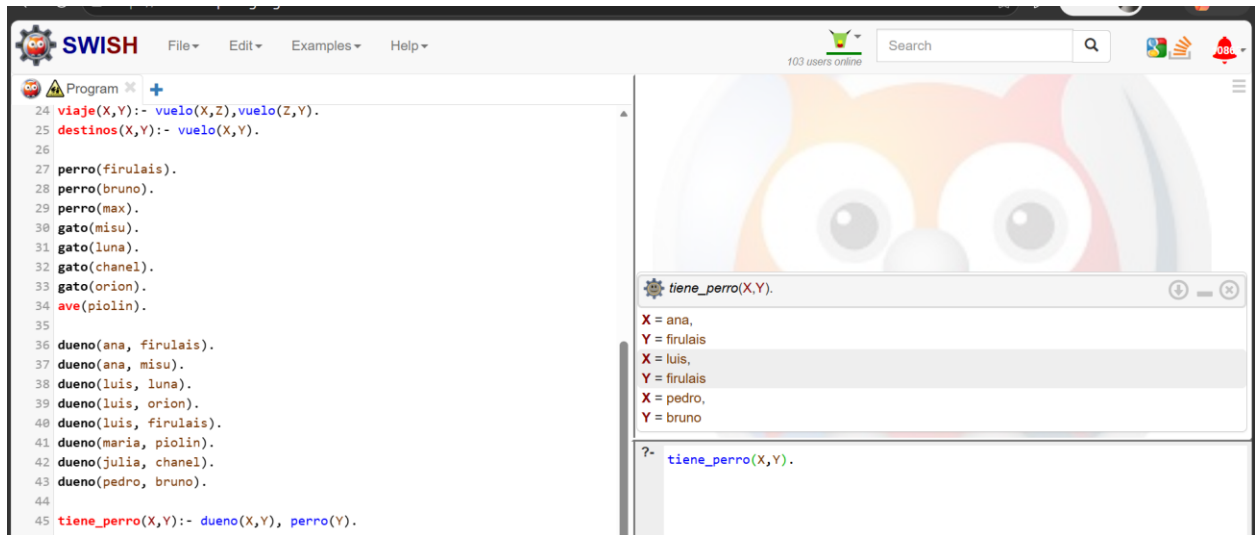


Define una regla destinos que devuelva todos los destinos alcanzables directamente desde una ciudad.

```
destinos(X,Y):- vuelo(X,Y).
```

Define una regla que determine si una persona tiene un perro.

- Ahora, encuentra los dueños de perros.



The screenshot shows the SWISH Prolog IDE. The left pane contains a Prolog program with the following code:

```
24 viaje(X,Y):- vuelo(X,Z),vuelo(Z,Y).
25 destinos(X,Y):- vuelo(X,Y).
26
27 perro(firulais).
28 perro(bruno).
29 perro(max).
30 gato(misu).
31 gato(luna).
32 gato(chanel).
33 gato(orion).
34 ave(piolin).
35
36 dueno(ana, firulais).
37 dueno(ana, misu).
38 dueno(luis, luna).
39 dueno(luis, orion).
40 dueno(luis, firulais).
41 dueno(maria, piolin).
42 dueno(julia, chanel).
43 dueno(pedro, bruno).
44
45 tiene_perro(X,Y):- dueno(X,Y), perro(Y).
```

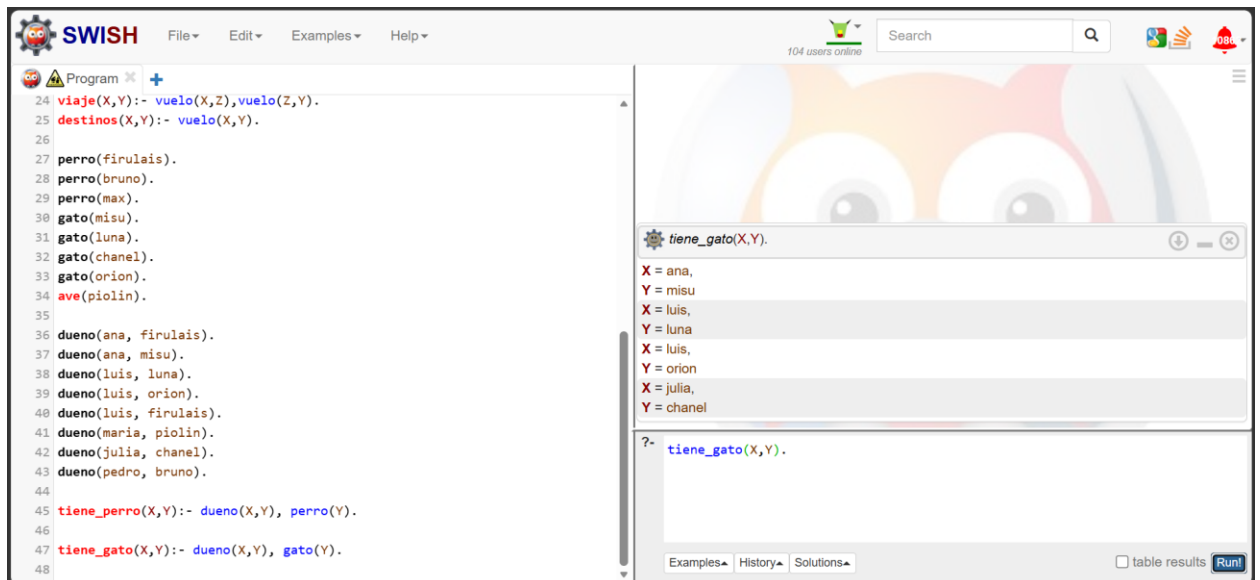
The right pane shows the execution results for the query `tiene_perro(X,Y).`:

```
X = ana,
Y = firulais
X = luis,
Y = firulais
X = pedro,
Y = bruno
```

Below the results, the query `?- tiene_perro(X,Y).` is shown.

Define una regla que determine si una persona tiene un gato.

Ahora, encuentra los dueños de gatos.



The screenshot shows the SWISH Prolog IDE. The left pane contains the same Prolog program as in the previous screenshot, but with an additional rule added at line 47:

```
45 tiene_perro(X,Y):- dueno(X,Y), perro(Y).
46
47 tiene_gato(X,Y):- dueno(X,Y), gato(Y).
```

The right pane shows the execution results for the query `tiene_gato(X,Y).`:

```
X = ana,
Y = misu
X = luis,
Y = luna
X = luis,
Y = orion
X = julia,
Y = chanel
```

Below the results, the query `?- tiene_gato(X,Y).` is shown.

Define una regla que determine si una persona tiene multiples mascotas.

Ahora, encuentra los dueños de multiple tipos de mascota.

```
 multiples_mascotas(X):- dueño(X,Y), dueño(X,Z), Y\=Z.
```



The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
26
27 perro(firulais).
28 perro(bruno).
29 perro(max).
30 gato(misu).
31 gato(luna).
32 gato(chanel).
33 gato(orion).
34 ave(piolin).
35
36 dueño(ana, firulais).
37 dueño(ana, misu).
38 dueño(luis, luna).
39 dueño(luis, orion).
40 dueño(luis, firulais).
41 dueño(maria, piolin).
42 dueño(julia, chanel).
43 dueño(pedro, bruno).
44
45 tiene_perro(X,Y):- dueño(X,Y), perro(Y).
46
47 tiene_gato(X,Y):- dueño(X,Y), gato(Y).
48
49 multiples_mascotas(X):- dueño(X,Y), dueño(X,Z), Y\=Z.
50
```

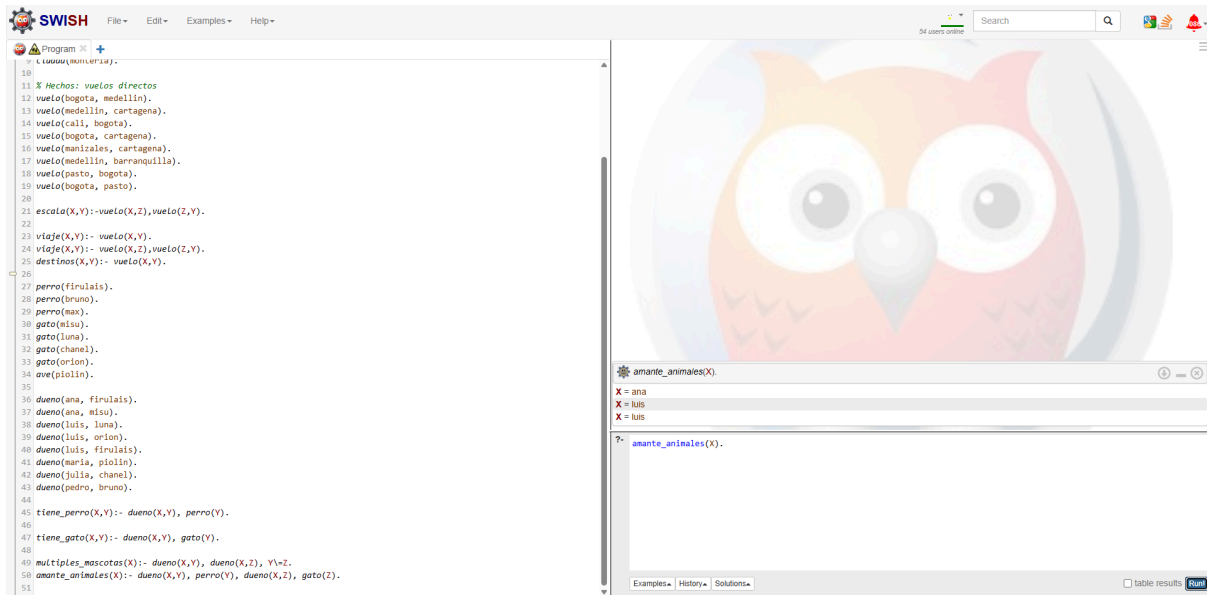
The right pane shows the execution results for the query `multiples_mascotas(X).`. The results are:

```
X = ana
X = ana
X = luis
X = luis
X = luis
X = luis
X = luis
X = luis
X = luis
```

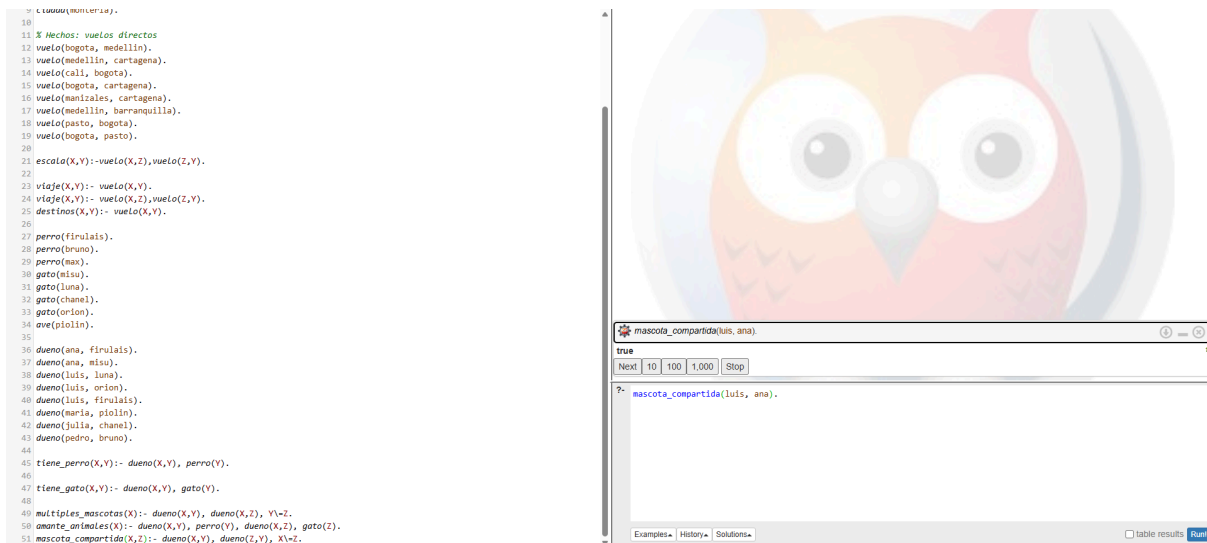
The bottom of the interface shows tabs for Examples, History, and Solutions, along with a checkbox for "table results" and a "Run" button.

- Define una regla amante_animales para identificar dueños que tienen tanto perro como gato.

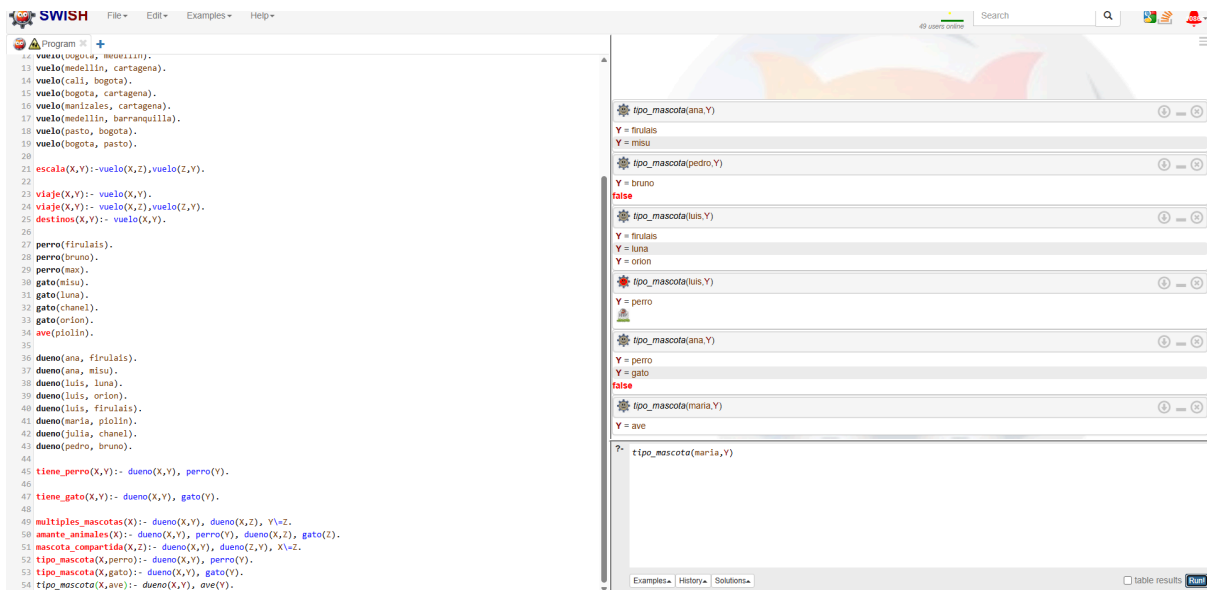
- Define una regla amante_animales para identificar dueños que tienen tanto perro como gato.



- Define una regla mascota_compartida que indique si dos personas comparten mascota.



- Define una regla tipo_mascota que asocie una persona con el tipo de mascota que tiene (perro, gato, ave, etc.).



Ejercicio 3 - Predicados con condiciones aritméticas

- Define una regla reprueba, que sea cierta si un estudiante tiene menor a 3.0.

```
7 reprueba(X,Y):- nota(X,Y), Y < 3.
8
```

- Define una regla aprueba, que sea cierta si un estudiante tiene nota mayor o igual a 3.0.

```
aprueba(X,Y):- nota(X,Y), Y >= 3.
```

- Define una regla rango, donde rango(X, Min, Max) sea cierto si la nota de un estudiante X está entre Min y Max.

```
rango(Y,X,Min,Max):- nota(Y,X), X >= Min, X <= Max.
```


- Define una regla clasificacion que asigne una categoría a cada estudiante según la nota:
 - 0.0 – 2.9 → reprobado
 - 3.0 – 3.9 → aprobado
 - 4.0 – 4.4 → notable
 - 4.5 – 5.0 → excelente

```
clasificacion(Est,C):- nota(Est,N), (N >= 0.0, N< 2.9 -> C = reprobado ; N >=3.0, N<3.9 -> C = aprobado;
  N >=4.0, N < 4.4 -> C = notable; N >=4.5, N < 5.0 -> C= excelente).
```

- Define la regla distancia((X1, Y1), (X2, Y2), D), que calcule la distancia euclidiana entre dos puntos

```
distancia(punto(X1, Y1), punto(X2, Y2), D) :- D is sqrt((X2 - X1)**2 + (Y2 - Y1)**2).
```

- Define la regla

distancia_total([P1, P2 | Resto], Total), que calcule la suma de distancias entre una lista puntos.

```
distancia_total([], 0).
distancia_total([_], 0).
distancia_total([P1, P2 | Resto], Total) :- distancia(P1, P2, D), distancia_total([P2 | Resto], DistanciaResto),
  Total is D + DistanciaResto.
```

Ejercicio 4 - Recursión simple

Define la regla fibonacci(X, N), donde X el el numero_x de la serie de fibonacci y F el valor correspondiente de la serie, uso de "is" para asignar valores a variables.

```
fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(X, F) :- X > 1, X1 is X - 1, X2 is X - 2, fibonacci(X1, F1), fibonacci(X2, F2), F is F1 + F2.
```

Ejercicio 5 - 4 color problem

Faltaria definir la regla `adjacent` la cual valide la adyacencia valida.

```
1 color(red).
2 color(blue).
3 color(yellow).
4 color(green).
5 adjacent(X,Y):- color(X), color(Y), X\= Y.
6
7 map(A, B, C, D, E) :-
8     adjacent(A, B), adjacent(A, D), adjacent(A, E),
9     adjacent(B, C), adjacent(B, D), adjacent(B, E),
10    adjacent(C, D), adjacent(C, E),
11    adjacent(D, E).
```

Ejercicio 6 - Listas

- Escribe una regla que verifique si un elemento es miembro de una lista.
- Escribe una regla "longitud" que dada una lista, calcule su longitud.
- Escribe una regla "concatena" que dadas dos listas, las concatene en otra.
- Escribe una regla que permite obtener en una lista los N elementos de las serie de fibonacci, usa la regla predifina `append(L1,L2,L3)`, que concatena L1 y L2, en L3.
- Escribe una regla que determine el reverso de una lista.
- Escribe una regla que determine si una lista es palindroma

```
miembro(X, [X|_]).
miembro(X, [_|T]) :- miembro(X, T).

longitud([], 0).
longitud([_|T], L) :- longitud(T, L1), L is L1 + 1.

concatena([], L, L).
concatena([X|L1], L2, [X|L3]) :- concatena(L1, L2, L3).

reverso([], []).
reverso([H|T], R) :- reverso(T, RT), append(RT, [H], R).

palindromo(L) :- reverso(L, L).|
```